

Resumen de Capacitación JavaScript

Unidad 2: Cadenas

En esta unidad se aprendió a trabajar con cadenas de texto en JavaScript. Se estudiaron propiedades y métodos como `length`, `toUpperCase()`, `toLowerCase()`, `substring()` y `at()`. También se aprendió a concatenar cadenas, utilizar plantillas literales (template literals) e interpolar variables dentro de textos.

Last updated: April 2026

¡Felicidades por completar el capítulo Cadenas I. Profundizaremos en las cadenas en un capítulo futuro, por ahora, asegúrate de repasar los conceptos clave de este capítulo antes de pasar al siguiente!

¡Este es tu primer Resumen del capítulo! Puedes guardarlo en tus notas haciendo clic en el ícono + en la parte superior derecha del resumen.

Dato curioso: ¿Sabías por qué usamos las palabras **bug** y **debug** en programación?

El término *bug* era en realidad argot de Ingeniería para fallos mucho antes de que existieran las computadoras; Thomas Edison lo usó en sus notas técnicas ya en la década de 1870.

Sin embargo, la palabra se hizo legendaria en la informática en 1947. Los operadores del equipo de Grace Hopper en Harvard encontraron una polilla real atrapada en un relé de la computadora Harvard Mark II. Pagaron el insecto en su libro de registro con la leyenda: "Primer caso real de bug encontrado."

Aunque no inventaron la palabra, su sesión literal de *debugging* convirtió una vieja broma de ingeniería en una parte permanente de la historia de la programación. ¡Cada vez que

Resumen del capítulo 

Resumen del capítulo

- Puedes crear cadenas con `"` o `'`.
- `.length` es una propiedad que te da la longitud de una cadena.
- `.toUpperCase()` es una función que convierte la cadena a mayúsculas.
- `.toLowerCase()` es una función que convierte la cadena a minúsculas.
- Los paréntesis `()` en las llamadas a funciones son obligatorios. `.length` es una propiedad que ya está precalculada; por lo tanto, no necesita paréntesis.
- `console.log(...)` se usa para depuración y NO es un reemplazo de `return`.
- Los corchetes `[index]` se usan para acceder a un índice específico de una cadena.
- El índice comienza en 0. Así que el primer carácter es el índice 0.
- Puedes combinarlo con la longitud de una cadena para obtener otro carácter en otra posición.
- El método `.at()` te permite leer un carácter en un índice (que también puede ser negativo).
- Una subcadena es una parte o una porción de una cadena.
- `string.substring(indexStart, indexEnd)` se usa para devolver una porción de la cadena.
- `indexStart`: la posición del primer carácter que te gustaría **incluir**.
- `indexEnd`: la posición del primer carácter que te gustaría **ignorar**.
- el argumento `indexEnd` es opcional, lo que significa que puedes omitirlo.
- El operador `+` se usa para sumar 2 números.
- El operador `+` se usa para concatenar 2 cadenas.
- Una cadena de plantilla es una cadena creada con el carácter de tilde grave: ```.
- Las cadenas de plantilla pueden abarcar varias líneas.
- Las cadenas de plantilla soportan la interpolación con la sintaxis `${variableName}`.

Unidad 3: Variables

En esta unidad se estudiaron las conversiones entre números y cadenas utilizando métodos como `toString()` y `Number.parseInt()`. También se revisó el significado de `NaN`, el operador módulo `%` y funciones matemáticas como `Math.round()`, `Math.floor()` y `Math.ceil()` para realizar operaciones de redondeo.

Resumen del capítulo

Last updated December 2025

¡Buen trabajo! Lenta pero seguramente estás aprendiendo JavaScript.

¿Has notado que estamos usando punto y coma en nuestro código? El punto y coma es opcional en JavaScript. Puedes decidir si incluirlo o no. La razón por la que existen es que el código JavaScript que escribas será minificado una vez que implementes tu sitio web en producción. ¡Esto se cubrirá con más detalle en un capítulo posterior!

Los desafíos y proyectos se volverán más interesantes a medida que aprendamos más sobre JavaScript. ¡A continuación, las variables!

Resumen del capítulo

- Convertir de número a cadena: `value.toString()`
- `NaN` significa Not a Number (No es un número)
- `NaN` es a menudo una señal de un error.
- Convertir de cadena a número `Number.parseInt(value, 10)`.
- `Number.parseInt()` es el nombre de la función que estás llamando.
- `10` es la **base** que debes especificar.
- Asegúrate de especificar siempre la base para evitar sorpresas desagradables.
- El operador de módulo (`%`) devuelve el resto de la división entre 2 números.
- `Math.round()` redondea un número al número entero más cercano; si la parte decimal es 0.5 o superior, redondea hacia arriba, de lo contrario, redondea hacia abajo.
- `Math.floor()` siempre "redondea hacia abajo" (trunca) el número al siguiente entero inferior.
- `Math.ceil()` siempre "redondea hacia arriba" el número al siguiente entero superior.

Unidad 4: Condicionales

En esta unidad se aprendió a declarar variables utilizando `let` y `const`. Se analizó la diferencia entre ambas formas de declaración, cuándo utilizar cada una y cómo influye el ámbito de las variables dentro del programa. Además, se revisó el uso de `var` y sus diferencias con las formas modernas de declaración.

Resumen del capítulo

Last updated January 2022

Definir variables nos permitirá trabajar en desafíos más interesantes, especialmente después de que aprendamos sobre arrays (cubierto en el capítulo 6).

Las variables definidas con `let` y `const` tienen "ámbito de bloque" (block scoped). Esto se explicará la primera vez que encontremos un bloque dentro de una función.

Si vienes de learnprogramming.online, habrás notado que no mencionamos `const` en absoluto en ese curso. Eso fue a propósito para hacer tu vida un poco más fácil mientras aprendías programación. Ahora que sabes sobre `const`, intenta usarlo cuando sea posible, aunque como puedes ver, todo también funciona con `let`. Simplemente `const` te dará la garantía de que cuando defines algo como un array (por ejemplo) se mantendrá como un array.

Resumen

- Cuando usas una variable por primera vez en JavaScript, necesitas declararla con `let` o `const`.
- Usa `let` para variables a las que necesitarás reasignar más adelante (como cambiar su valor).
- Usa `const` para variables a las que no necesitarás reasignar más adelante.
- Las variables declaradas con `const` **no** son constantes. Veremos por qué más adelante en este curso.
- Las variables declaradas con `const` no pueden ser reasignadas, por lo que no puedes tener el `=` junto a ese nombre de variable después de declararla.
- Si ves `var`, es de la versión antigua de JavaScript. Puedes convertirlo a `let` (a veces `const` si la variable no es reasignada).

Unidad 5: Funciones

En esta unidad se estudiaron las estructuras condicionales para controlar el flujo de ejecución de los programas. Se trabajó con las sentencias `if` y `else`, además del operador ternario, que permite escribir condiciones de forma más compacta y sencilla cuando se manejan decisiones simples.

Last updated April 2026

¡Buen trabajo!

El operador ternario

Las condiciones `if` cortas a veces pueden escribirse usando el operador ternario. El operador ternario es una forma corta de escribir una declaración `if ... else`.

Volvamos a la solución del desafío anterior:

```
function evenOrOdd(number) {  
  if (number % 2 === 0) {  
    return "even";  
  }  
  return "odd";  
}
```

Puede escribirse usando el operador ternario de la siguiente manera:

```
function evenOrOdd(number) {  
  return (number % 2 === 0) ? "even" : "odd";  
}
```

El operador ternario tiene la siguiente sintaxis:

```
condition ? expressionWhenTrue : expressionWhenFalse
```

Si bien el operador ternario puede hacer que tu código sea más conciso, también puede hacerlo menos legible. Generalmente te recomendamos evitarlo y optar por un código

Resumen del capítulo

⚡ Acerca del instructor

Una presentación tardía: Hola 🙌, soy [Jad Joubran](#). Soy el escritor de este curso, creador y mantenedor de la plataforma donde estás estudiando.



Conclusiones

La capacitación permitió adquirir conocimientos fundamentales de JavaScript relacionados con el manejo de cadenas, números, variables y estructuras condicionales. Estos conceptos constituyen una base sólida para el desarrollo de aplicaciones web y serán utilizados en las siguientes fases del proyecto para construir un dashboard interactivo y funcional.